UNIVERSIDAD DE ANTIOQUIA

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

Arquitectura de Software

Laboratorio 3

Construcción e integración de un servidor MCP con Spring Boot, Spring AI, MySQL, Claude Desktop y Google ADK

Sara Mesa Gómez

Profesor: Diego José Luis Botia

2026

1. Introducción

Model Context Protocol (MCP) es un protocolo abierto, presentado por Anthropic en 2024, que estandariza cómo una aplicación de inteligencia artificial descubre y usa herramientas externas. Un servidor MCP pública operaciones de negocio con nombre, descripción y esquema de parámetros, y cualquier cliente compatible puede invocarlas sin conocer el código que las implementa.

En este laboratorio se construyó un servidor MCP con Spring Boot y Spring AI que administra tareas almacenadas en MySQL. Ese mismo servidor se conectó a dos clientes distintos: Claude Desktop, mediante el puente `mcp-remote`, y un agente programado con Google Agent Development Kit (ADK) para Java que usa el modelo Gemini. El objetivo fue comprobar que un único conjunto de herramientas sirve a varios clientes sin modificar el servidor.

Además se resolvieron las actividades 10 a 16 de la guía y el ejercicio propuesto: un chatbot web para hacer pedidos de supermercado a domicilio, integrado con un segundo servidor MCP conectado a MySQL. Este informe describe las herramientas usadas, la arquitectura, el procedimiento con sus evidencias, los problemas encontrados y las conclusiones.

2. Objetivos

Objetivo general

Construir un servidor MCP con Spring Boot y Spring AI que publique operaciones sobre una base de datos MySQL, e integrarlo con distintos clientes de inteligencia artificial para que un usuario pueda consultar y modificar datos reales mediante lenguaje natural.

Objetivos específicos

- Implementar un servidor MCP con anotaciones `@Tool` y transporte Streamable HTTP, con persistencia en MySQL mediante Spring Data JPA.
- Verificar el handshake del protocolo y el catálogo de herramientas con curl y MCP Inspector.
- Conectar Claude Desktop al servidor local usando el puente STDIO a HTTP `mcp-remote`.
- Crear un agente con Google ADK para Java que descubra e invoque las herramientas del mismo servidor desde Gemini.
- Ampliar el servidor con las herramientas `contarTareas` y `listarCompletadas` y con validación de fechas ISO, y cubrir la lógica con pruebas automatizadas.
- Desarrollar un chatbot de pedidos de supermercado a domicilio como cliente de un segundo servidor MCP conectado a MySQL.
- Aplicar buenas prácticas de manejo de secretos y de seguridad para herramientas expuestas por HTTP.

3. Herramientas de software empleadas

Herramienta	Uso en el laboratorio
Java 21 y Spring Boot 3.4.7	Lenguaje y framework de los servidores MCP y del chatbot.

Herramienta	Uso en el laboratorio
Spring AI 1.1.4	Implementa el servidor MCP (`spring-ai-starter-mcp-server-webmvc`), el cliente MCP del chatbot y la integración con Gemini.
Maven Wrapper	Gestión de dependencias, pruebas (`mvnw clean test`) y ejecución (`mvnw spring-boot:run`).
MySQL 8 y MySQL Workbench	Base de datos de las tareas (`labmcp`) y del supermercado (`supermercado_mcp`); Workbench para crear los esquemas y verificar los cambios.
H2 Database	Base en memoria usada solo durante las pruebas, para no depender de MySQL.
JUnit 5 y Spring Boot Test	Pruebas de la lógica de negocio y del protocolo MCP.
curl	Envío manual del mensaje `initialize` para verificar el protocolo.
MCP Inspector 0.12.0	Cliente gráfico para listar y ejecutar las herramientas del servidor.
Node.js 22 y `mcp-remote`	Ejecutan MCP Inspector y el puente STDIO a HTTP que necesita Claude Desktop.
Claude Desktop	Host MCP: conversa con el modelo Claude y usa las herramientas del servidor.
Google ADK para Java (0.6.0 y 1.0.0)	Agente `TareaAgent` y su interfaz web de pruebas en el puerto 8080.
Gemini (`gemini-3.5-flash`)	Modelo de lenguaje del agente ADK y del chatbot del supermercado.
Git y GitHub	Control de versiones y repositorio remoto del proyecto.
Visual Studio Code	Editor y terminal integrada para el desarrollo y la ejecución local.

4. Arquitectura propuesta

Un único servidor MCP atiende a dos clientes distintos sin cambiar una línea de su código. La solución tiene cuatro roles:

- **Host:** la aplicación que contiene la conversación y el modelo de lenguaje, en este caso Claude Desktop o el agente ADK.

- **Cliente MCP:** el componente dentro del host que negocia el protocolo, consulta el catálogo de herramientas y envía las llamadas `tools/call`.
- **Servidor MCP:** la aplicación Spring Boot que anuncia las herramientas y traduce cada llamada a un método Java.
- **Persistencia:** MySQL, accedido mediante Spring Data JPA.

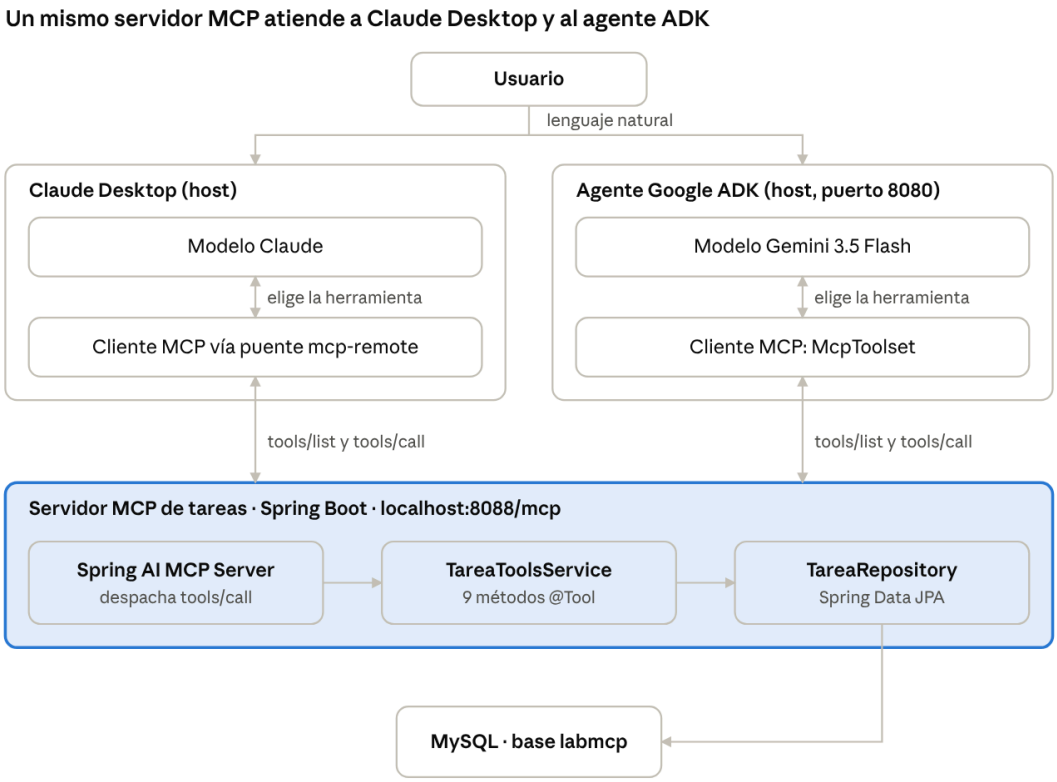


Figura A. Arquitectura: dos hosts, un servidor MCP y MySQL

Cada host contiene su propio modelo y su cliente MCP. Ambos llegan al mismo endpoint `/mcp`, y solo el servidor accede a MySQL.

Servidor de tareas. La aplicación está organizada por capas: la entidad `Tarea`, el repositorio `TareaRepository` y el servicio `TareaToolsService`, cuyos métodos anotados con `@Tool` se registran como herramientas mediante un `ToolCallbackProvider`. El endpoint es `http://localhost:8088/mcp` con transporte Streamable HTTP. Publica 9 herramientas: las 7 de la guía y 2 agregadas en las actividades 10 y 11.

Herramienta	Función
`listarTareas`	Devuelve todas las tareas.
`listarPendientes`	Devuelve las tareas con `completado = false`.
`listarCompletadas`	Devuelve las tareas con `completado = true` (actividad 11).

Herramienta	Función
`buscarTareas`	Busca tareas cuyo título contiene un fragmento, sin distinguir mayúsculas.
`crearTarea`	Crea una tarea pendiente con título y fecha ISO.
`marcarCompletada`	Marca como completada la tarea indicada.
`actualizarTarea`	Cambia el título o la fecha de una tarea.
`eliminarTarea`	Elimina la tarea indicada.
`contarTareas`	Devuelve el número total de tareas (actividad 10).

Componentes y puertos. Cada componente corre en su propio proceso y puerto:

Componente	Carpeta	Puerto
Servidor MCP de tareas	raíz del repositorio	8088
Agente Google ADK	`agente-mcp/`	8080
Servidor MCP del supermercado	`supermercado/mcp-server/`	8090
Chatbot del supermercado	`supermercado/chatbot/`	8081

Manejo de secretos. La contraseña de MySQL (`PWD_UDEA`) y la clave de Gemini (`GOOGLE_API_KEY`) no están en el código. Se leen de variables de entorno o de un archivo `.env` local que Git ignora; el repositorio solo incluye la plantilla `.env.example`. Los módulos Spring lo cargan con `spring.config.import` y el agente ADK lo lee al iniciar.

5. Procedimiento

El laboratorio se desarrolló en el orden de la guía: base de datos, servidor, pruebas del protocolo, Claude Desktop y agente ADK. Cada parte se verificó contra MySQL antes de pasar a la siguiente.

Preparación del entorno

Maven usa la variable `JAVA_HOME`, que apuntaba al JDK 17; se cambió al JDK 21 que exige el proyecto. Las credenciales se configuraron copiando `.env.example` como `.env` en la raíz del repositorio y completando `PWD_UDEA` con la contraseña de MySQL y `GOOGLE_API_KEY` con una clave generada en Google AI Studio.

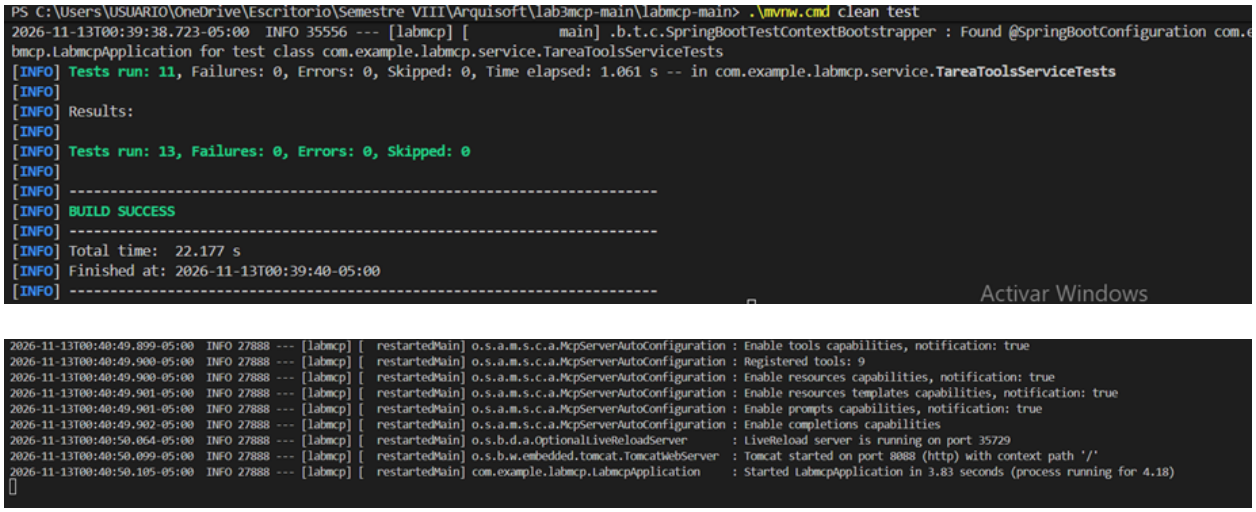
Parte A: base de datos

El script `src/main/resources/schema.sql` crea la base `labmcp`, la tabla `tarea` (id, titulo, fecha, completado, version) e inserta tres tareas de ejemplo. Se ejecutó una sola vez desde MySQL Workbench con File, Open SQL Script y el botón Execute, porque cada ejecución vuelve a insertar los datos de ejemplo.

Parte B: servidor MCP con Spring Boot y Spring AI

El proyecto base se alineó con la guía. Las herramientas se renombraron de `listTareas` y `markComplete` a `listarTareas` y `marcarCompletada`, y se agregaron `listarPendientes`, `buscarTareas` y `actualizarTarea`. La configuración define el protocolo `STREAMABLE`, el endpoint `/mcp` y el puerto 8088.

Con `.\mvnw.cmd clean test` se ejecutaron las 13 pruebas automatizadas sin fallos. Con `.\mvnw.cmd spring-boot:run` el servidor arrancó y registró las 9 herramientas (`Registered tools: 9`).



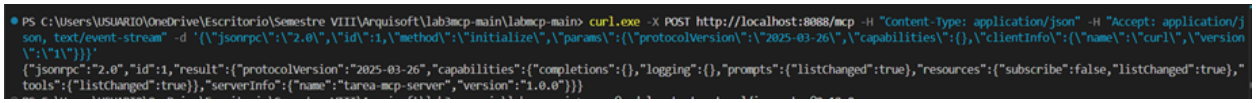
[Figura 1. Ejecución de las pruebas (13 pruebas, BUILD SUCCESS) y arranque del servidor con Registered tools: 9]

Parte C: prueba del protocolo MCP

Negociación con curl. Se envió el mensaje `initialize` de JSON-RPC 2.0 al endpoint:

```
curl.exe -X POST http://localhost:8088/mcp -H "Content-Type: application/json" -H "Accept: application/json, text/event-stream" -d '{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"initialize\",\"params\":{\"protocolVersion\":\"2025-03-26\",\"capabilities\":{},\"clientInfo\":{\"name\":\"curl\",\"version\":\"1\"}}}'
```

El servidor respondió con la versión de protocolo, sus capacidades (tools, resources, prompts, logging y completions) y su identificación `tarea-mcp-server` versión 1.0.0.

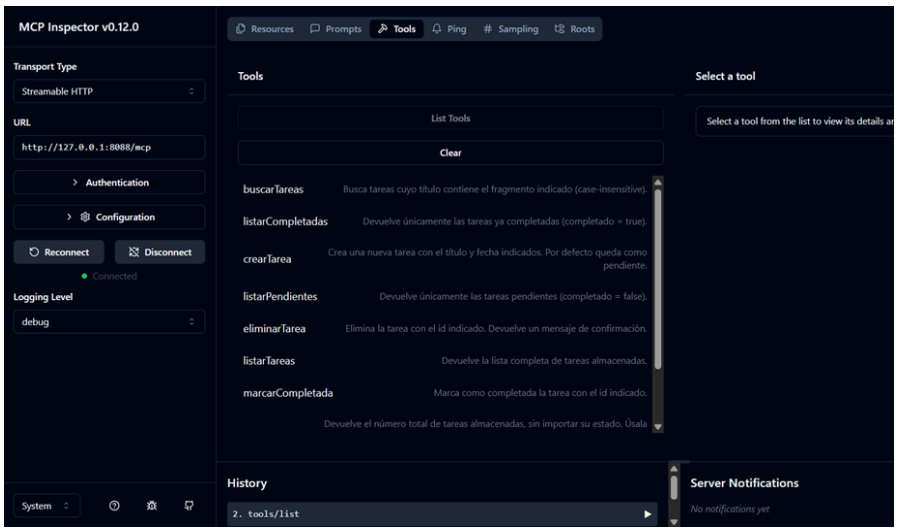


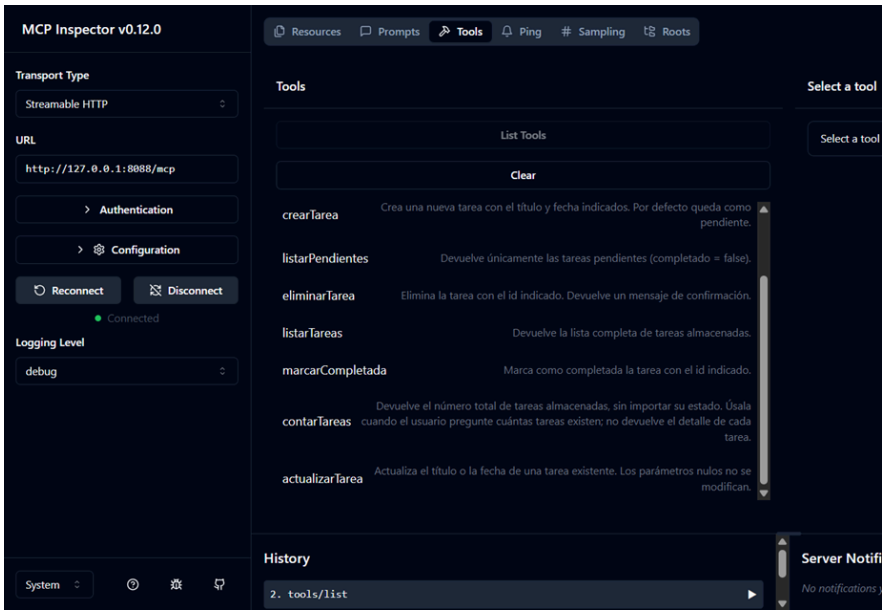
[Figura 2. Respuesta del servidor al mensaje initialize]

MCP Inspector. Se ejecutó `npx @modelcontextprotocol/inspector@0.12.0` y se conectó con transporte Streamable HTTP a `http://127.0.0.1:8088/mcp`. La opción List Tools mostró las 9 herramientas con sus descripciones.

```
Starting MCP inspector...
⚙️Proxy server listening on port 6277
🔍 MCP Inspector is up and running at http://127.0.0.1:6274 🚀
Received POST message for sessionId undefined
New streamable-http connection
Query parameters: [Object: null prototype] {
  url: 'http://127.0.0.1:8088/mcp',
  transportType: 'streamable-http'
}
Connected to Streamable HTTP transport
Connected MCP client to backing server transport
Created streamable web app transport 46b30027-fead-47e8-bdce-8fc55d4b0cc2
Received POST message for sessionId 46b30027-fead-47e8-bdce-8fc55d4b0cc2
Received GET message for sessionId 46b30027-fead-47e8-bdce-8fc55d4b0cc2
```

[Figura 3. Consola de MCP Inspector conectada por Streamable HTTP]



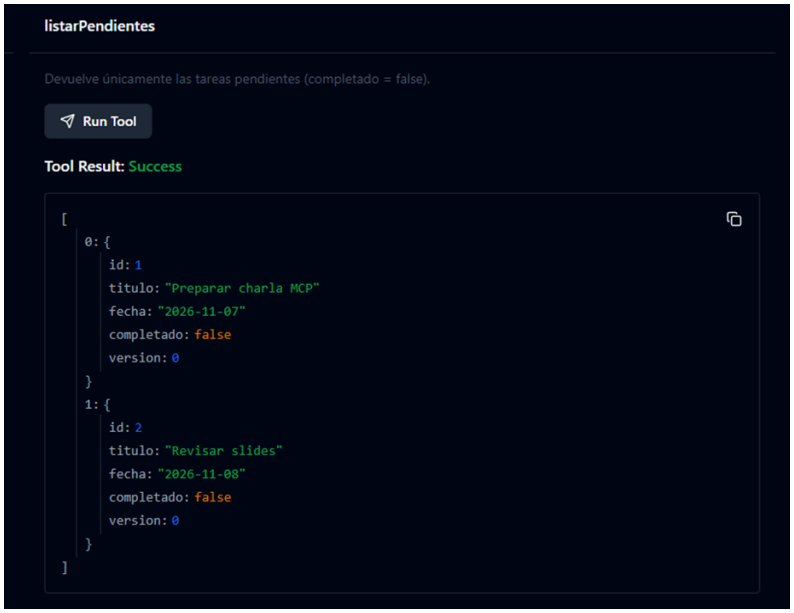


[Figura 4. Lista de herramientas en MCP Inspector]

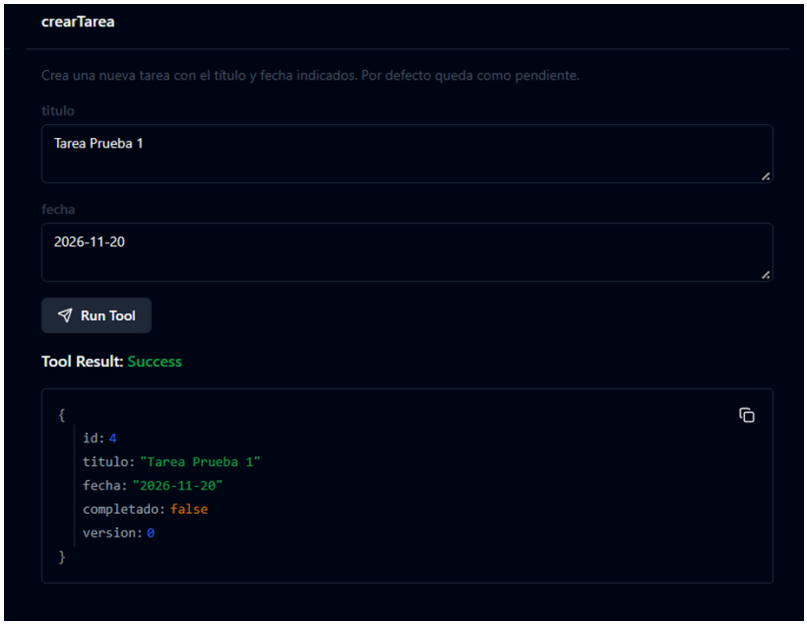
Prueba de las herramientas. Desde Inspector se ejecutaron las herramientas y cada cambio se comprobó en MySQL con:

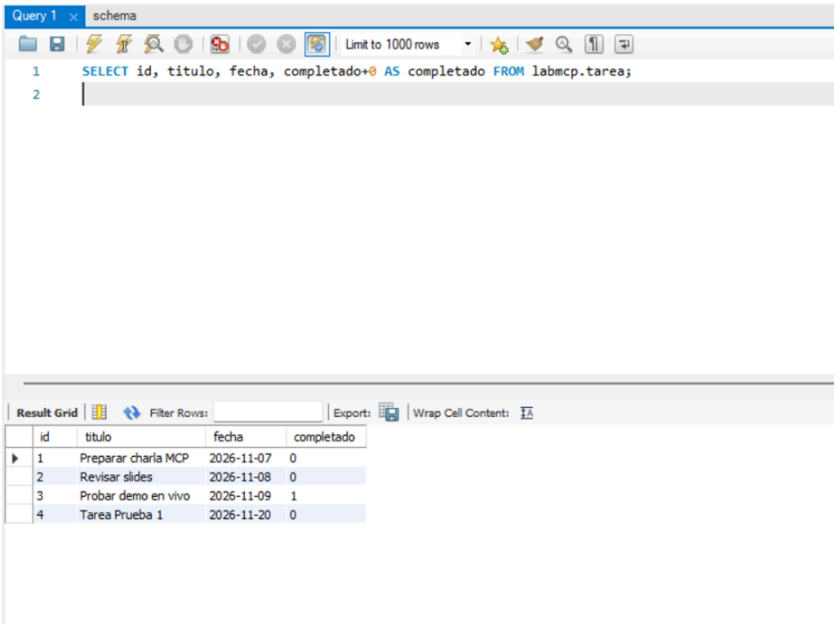
```
SELECT id, titulo, fecha, completado+0 AS completado FROM labmcp.tarea;
```

1. `listarPendientes` devolvió solo las tareas 1 y 2, que tienen `completado = false`.
2. `crearTarea` con título "Tarea Prueba 1" y fecha 2026-11-20 creó la tarea 4, y la fila apareció en MySQL.
3. `marcarCompletada` con id 4 cambió `completado` a 1 y subió la versión de la fila de 0 a 1, por el bloqueo optimista de JPA.



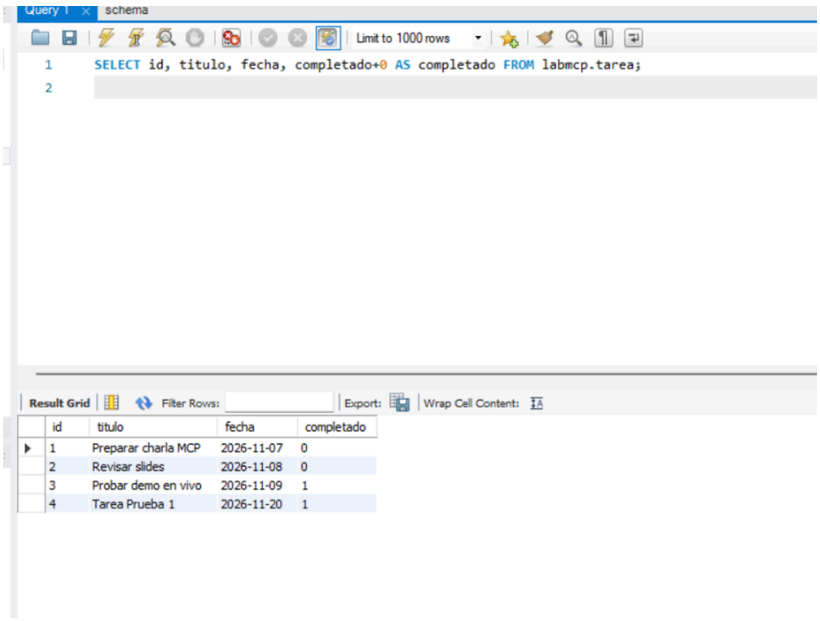
[Figura 5. Resultado de listarPendientes]





[Figura 6. crearTarea y fila nueva en MySQL]





[Figura 7. marcarCompletada y fila con completado = 1 en MySQL]

Parte D: Claude Desktop

Claude Desktop solo acepta servidores STDIO en su archivo de configuración, así que se usó `mcp-remote` como puente hacia el endpoint HTTP. En Desarrollador, Abrir archivo de configuración, se registró:

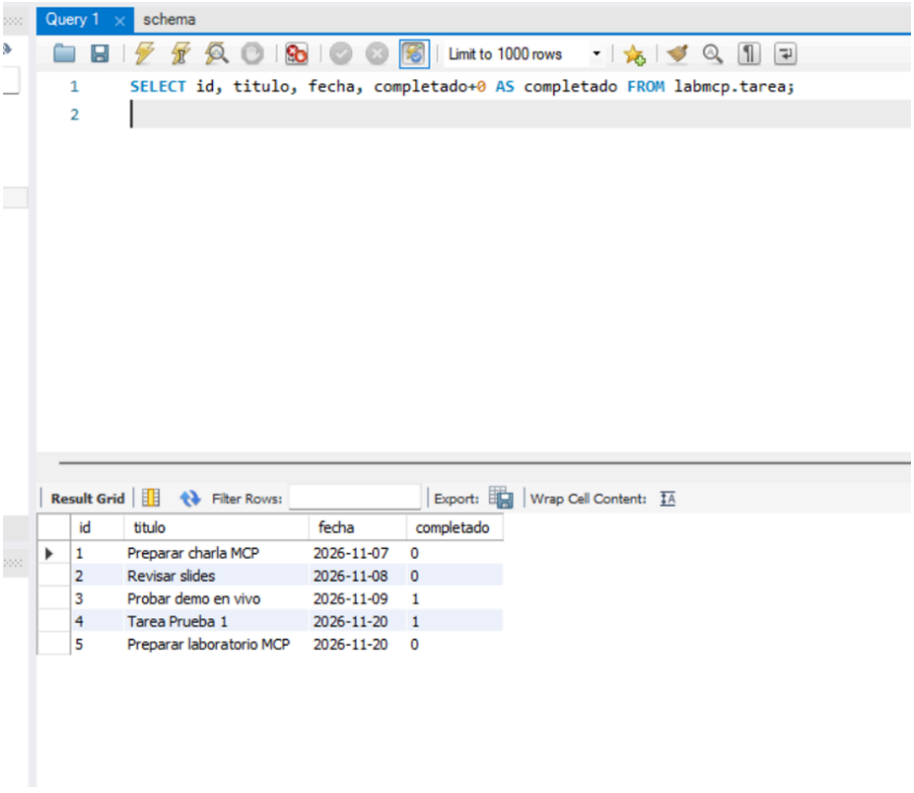
```
{
  "mcpServers": {
    "tareas-mcp": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "http://localhost:8088/mcp"]
    }
  }
}
```

El archivo no contiene la contraseña de MySQL ni la clave de Gemini, porque el cliente solo necesita la URL del servidor. Tras reiniciar Claude Desktop, el conector `tareas-mcp` apareció activo.

A la petición "Lista mis tareas pendientes", Claude invocó `listarPendientes` y respondió con las tareas 1 y 2, señalando que sus fechas ya habían pasado. A la petición "Crea una tarea llamada Preparar laboratorio MCP con fecha 2026-11-20", invocó `crearTarea` y la tarea 5 quedó registrada en MySQL.



[Figura 8. Conversación en Claude Desktop usando las herramientas de tareas-mcp]



[Figura 9. Tarea 5 creada desde Claude Desktop, vista en MySQL]

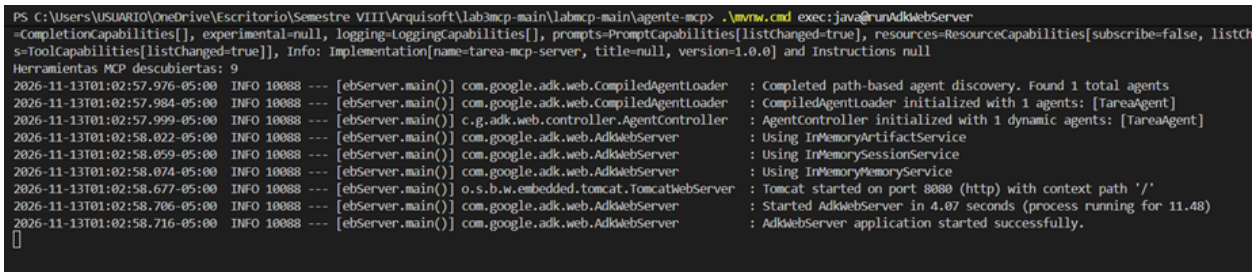
Parte E: agente con Google ADK para Java

El proyecto `agente-mcp` define la clase `TareaAgent`, que crea un `McpToolset` con `StreamableHttpServerParameters` apuntando a `http://localhost:8088/mcp`, descubre las herramientas al iniciar y las entrega a un `LlmAgent` con Gemini. Su instrucción le pide confirmar el identificador antes de eliminar una tarea.

Se ejecutó con:

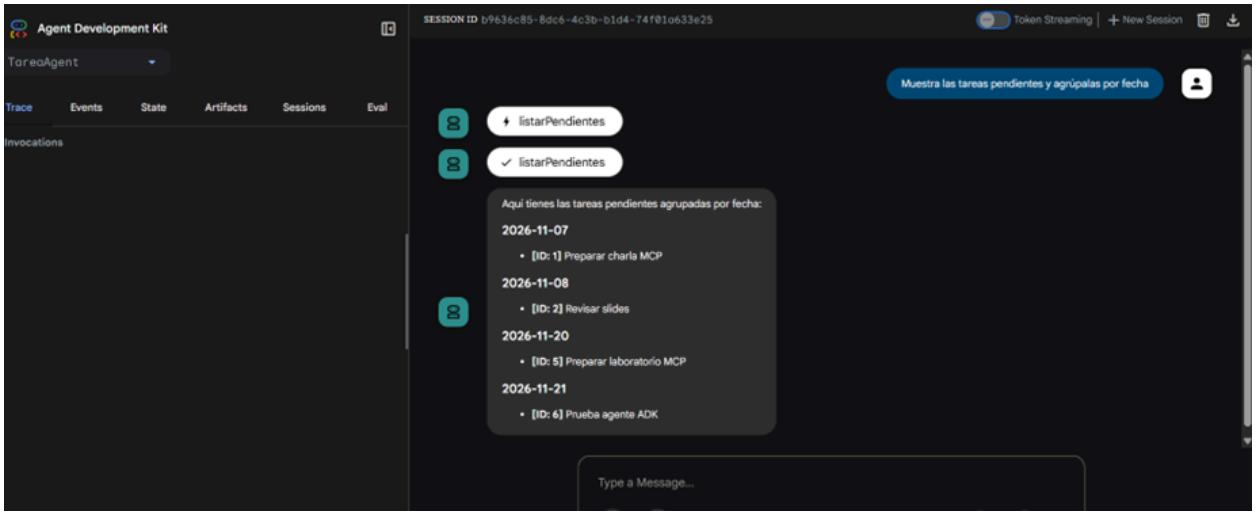
```
cd agente-mcp
.\mvnw.cmd clean install
.\mvnw.cmd exec:java@runAdkWebServer
```

La consola reportó `Herramientas MCP descubiertas: 9` y la interfaz ADK Web quedó disponible en http://localhost:8080.

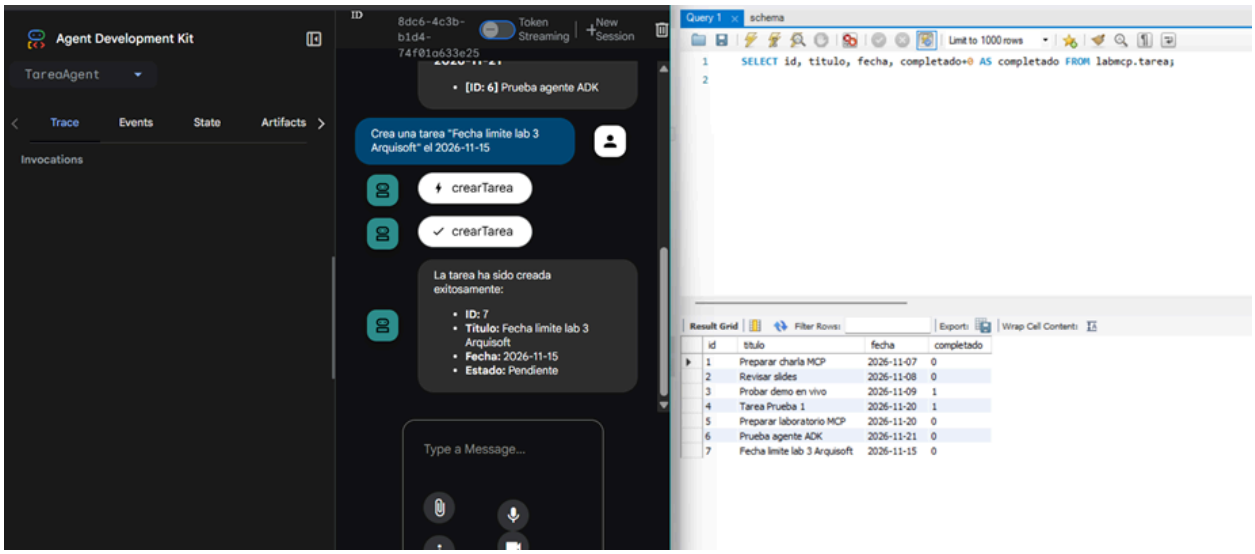


[Figura 10. Arranque del agente con 9 herramientas MCP descubiertas]

Ante "Muestra las tareas pendientes y agrúpalas por fecha", el agente invocó `listarPendientes` y agrupó el resultado por fecha. Después se le pidió crear la tarea "Fecha límite lab 3 Arquisoft" con fecha 2026-11-15: invocó `crearTarea` y la tarea 7 apareció en MySQL. La interfaz muestra cada llamada a herramienta como un evento.



[Figura 11. Agente ADK invocando listarPendientes]



[Figura 12. Agente ADK invocando crearTarea y fila nueva en MySQL]

6. Actividades para los estudiantes

Actividad 10: herramienta `contarTareas`

Se implementó en `TareaToolsService` y devuelve `repo.count()`. Su descripción MCP es: "Devuelve el número total de tareas almacenadas, sin importar su estado. Úsala cuando el usuario pregunte cuántas tareas existen; no devuelve el detalle de cada tarea".

El modelo no lee el código Java: solo ve el nombre, la descripción y el esquema JSON que publica `tools/list`. Por eso la descripción dice qué devuelve, cuándo usarla y en qué se diferencia de `listarTareas`. Así, ante "¿cuántas tareas tengo?", el modelo no trae la lista completa para contarla.

Actividad 11: herramienta `listarCompletadas`

Usa la misma consulta derivada que `listarPendientes`, `findByCompletado`, con el argumento en `true`. Las dos herramientas son complementarias: cada tarea aparece en una sola de las dos listas, y la suma de ambas coincide con `contarTareas`. Con los datos de ejemplo, `listarPendientes` devuelve las tareas 1 y 2, y `listarCompletadas` la tarea 3.

Actividad 12: validación de fecha ISO

`crearTarea` y `actualizarTarea` validan la fecha con `LocalDate.parse(valor, DateTimeFormatter.ISO_LOCAL_DATE)`. Este formateador comprueba el formato `yyyy-MM-dd` y también que la fecha exista, así que rechaza `20/11/2026`, `2026-11-5` y `2026-02-30`. La validación está en el servidor porque las instrucciones del modelo no son un control confiable. El error llega al cliente como resultado MCP con `isError: true`, y el modelo puede explicárselo al usuario.

Actividad 13: host, cliente MCP, servidor MCP y herramienta

La Figura A de la sección 4 muestra la diferencia entre los cuatro elementos: el host contiene al modelo y al cliente MCP; el cliente habla con el servidor por JSON-RPC; el servidor expone las herramientas, y cada herramienta es un método que accede a MySQL.

Elemento	Responsabilidad	En este laboratorio
Host	Contiene la conversación y el modelo, y decide qué permite.	Claude Desktop y el agente ADK
Cliente MCP	Negocia el protocolo, lista las herramientas y envía `tools/call`. Mantiene una conexión por servidor.	Interno del host; en Claude Desktop se apoya en `mcp-remote`
Servidor MCP	Anuncia sus capacidades y traduce cada llamada a código.	Aplicación Spring Boot en el puerto 8088
Herramienta	Una operación concreta con nombre, descripción y esquema de parámetros.	Cada método `@Tool` de `TareaToolsService`

Actividad 14: Claude Desktop y Google ADK como clientes del mismo servidor

Aspecto	Claude Desktop	Google ADK
Tipo de cliente	Aplicación de escritorio ya construida	Agente programado en Java
Modelo	Claude	Gemini
Conexión	Archivo JSON, con `mcp-remote` como puente STDIO a HTTP	`McpToolset` con `StreamableHttpServerParameters`
Código necesario	Ninguno	Clase `TareaAgent` y su POM
Instrucciones	Las que escribe el usuario en el chat	Definidas en `LlmAgent.instruction`
Interfaz	La de la propia aplicación	ADK Web en el puerto 8080

Se reutiliza todo el servidor sin cambios: las herramientas, sus descripciones y esquemas, las validaciones, JPA y MySQL. Solo cambia la configuración del lado del cliente. En las pruebas, ambos clientes eligieron las mismas herramientas (`listarPendientes` y `crearTarea`) ante peticiones equivalentes. Ese es el valor de MCP: una integración escrita una vez sirve a cualquier cliente compatible.

Actividad 15: prueba automatizada

- `TareaToolsServiceTests` cubre la lógica de negocio: conteo, listas complementarias, validación de título y de fechas, actualización parcial y eliminación.
- `McpProtocolTests` levanta el servidor en un puerto aleatorio y usa un cliente MCP real por Streamable HTTP. Comprueba el handshake, que se publiquen las 9 herramientas, una llamada válida y una inválida.

Las pruebas se ejecutan con `.\mvnw.cmd clean test` sobre H2 en memoria, sin necesitar MySQL. El resultado fue de 13 pruebas sin fallos (Figura 1).

Actividad 16: riesgo de exponer herramientas HTTP sin autenticación

Riesgo. Spring Boot escucha por defecto en todas las interfaces de red, y el endpoint `/mcp` no pide credenciales. Cualquier equipo que alcance el puerto 8088, por ejemplo en la misma red Wi-Fi de la universidad, puede listar las herramientas e invocar `eliminarTarea` con cada id. No necesita un modelo de lenguaje, le basta un `curl`. El resultado es pérdida o alteración de datos sin rastro de quién lo hizo.

Mitigación propuesta, por capas:

1. Limitar la exposición con `server.address=127.0.0.1` cuando el uso sea local. El servidor del supermercado ya lo aplica.
2. Autenticar y autorizar con Spring Security como servidor de recursos OAuth2, exigiendo un token con permisos por herramienta, por ejemplo separando lectura de eliminación.
3. Cifrar el tráfico con TLS en cualquier despliegue fuera del equipo local.
4. Validar el encabezado `Origin`, para impedir que una página web maliciosa alcance el servidor local mediante DNS rebinding.
5. Aplicar mínimo privilegio: publicar solo las herramientas necesarias y conectarse a MySQL con un usuario propio, no con `root`.
6. Registrar cada invocación con su usuario, sin incluir secretos, para poder auditar.

7. Ejercicio propuesto: chatbot de pedidos de supermercado a domicilio

Se construyó un chatbot web que permite a un cliente buscar productos, hacer un pedido a domicilio, consultarlo y cancelarlo conversando en lenguaje natural. El chatbot es un cliente MCP que usa las herramientas de un segundo servidor MCP conectado a MySQL. Ambos están en la carpeta `supermercado/`.

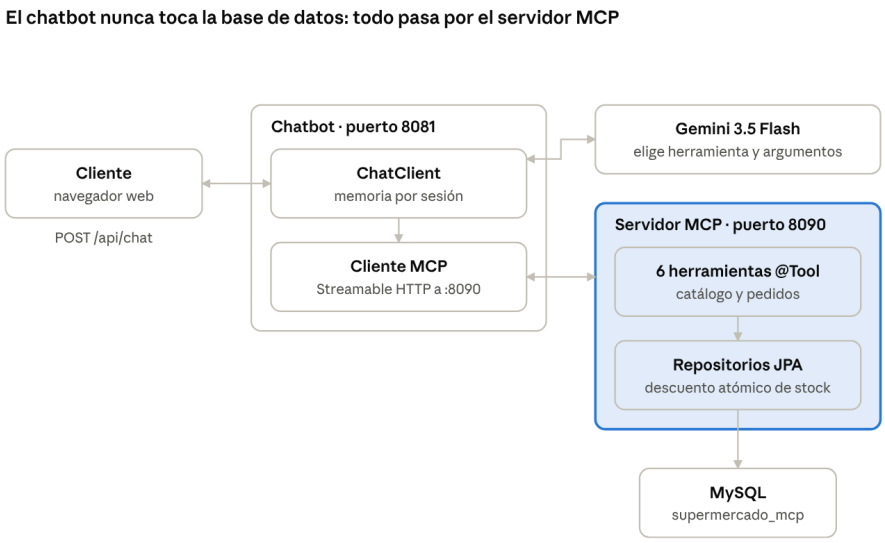


Figura B. Arquitectura del ejercicio propuesto

Gemini solo decide qué herramienta usar y con qué argumentos. Los precios, el stock y el registro del pedido los resuelve el servidor MCP en una transacción.

Paso 1. Definir el alcance

El cliente necesita explorar el catálogo, registrar un pedido con sus datos de entrega, consultarlo y cancelarlo. Se reutilizó el patrón de la guía (Spring Boot, Spring AI, `@Tool` y Streamable HTTP) y se eligió MySQL como motor de base de datos.

Paso 2. Modelar la base de datos

El script `mcp-server/src/main/resources/schema.sql` crea la base `supermercado_mcp` con tres tablas y carga 28 productos en 9 categorías. Usa `IF NOT EXISTS` e `INSERT IGNORE`, así que puede ejecutarse varias veces sin duplicar datos.

Tabla	Propósito	Detalle relevante
`producto`	Catálogo e inventario	`precio` es `DECIMAL` para no perder exactitud en dinero; un `CHECK` impide stock negativo.
`pedido`	Cliente, dirección, estado y total	`version` activa el bloqueo optimista.
`detalle_pedido`	Una línea por producto	Guarda `precio_unitario`, para que el pedido conserve su valor si el catálogo cambia.

Paso 3. Entidades y repositorios

`Producto`, `Pedido` y `DetallePedido` mapean las tablas con JPA.
`ProductoRepository.descontarStock` ejecuta una sola sentencia `UPDATE ... WHERE stock >= cantidad`, que devuelve 0 si no alcanza el inventario. Como la base de datos serializa esa sentencia, dos compras simultáneas no pueden vender la misma unidad.

Paso 4. Publicar las herramientas MCP

El servidor corre en el puerto 8090 y publica 6 herramientas:

Herramienta	Función
`listarCategorias`	Devuelve las categorías disponibles.
`buscarProductos`	Busca por nombre, categoría o ambos, con un máximo de 20 resultados.
`crearPedido`	Registra el pedido, descuenta el stock y calcula el total.
`consultarPedido`	Muestra un pedido, exigiendo su teléfono.
`listarPedidosCliente`	Muestra los 10 pedidos más recientes de un teléfono.
`cancelarPedido`	Cancela un pedido en estado `RECIBIDO` y repone el stock.

Paso 5. Reglas de negocio en el servidor

El modelo solo elige la herramienta y propone los argumentos, así que un error suyo no puede romper las reglas:

- El precio y el total se calculan en el servidor; los argumentos de `crearPedido` no incluyen precios.
- `crearPedido` es transaccional: si un producto no tiene stock, se revierte el pedido completo.
- Se validan nombre, teléfono, dirección y cantidades de 1 a 50.
- Consultar o cancelar exige el teléfono del pedido. Un teléfono equivocado recibe la misma respuesta que un pedido inexistente, para no revelar qué pedidos existen.
- El cambio de estado (preparación, despacho) no se publica como herramienta, porque corresponde al personal de la tienda y no al cliente.

Los métodos devuelven DTO en lugar de entidades JPA, para controlar qué datos recibe el modelo.

Paso 6. Construir el chatbot

El proyecto `chatbot` corre en el puerto 8081 e integra tres piezas:

1. **Modelo:** Gemini, con el starter `spring-ai-starter-model-google-genai`.
2. **Herramientas:** el starter `spring-ai-starter-mcp-client` se conecta al servidor al arrancar, ejecuta `initialize` y `tools/list`, y entrega las herramientas al modelo.
3. **Memoria:** `MessageChatMemoryAdvisor` conserva la conversación por sesión, para que el cliente pueda decir "agrega dos más".

`ChatController` expone `POST /api/chat` y `index.html` ofrece la interfaz web. Los mensajes se insertan como texto y nunca como HTML, para que una respuesta no pueda inyectar código en la página.

Paso 7. Instrucciones al modelo

El archivo `prompts/sistema.txt` define el comportamiento: consultar el catálogo antes de hablar de un producto, no inventar precios y mostrar un resumen antes de registrar nada. `crearPedido` solo se invoca cuando el cliente confirma ese resumen.

Paso 8. Seguridad

Los dos servicios escuchan solo en `127.0.0.1`, porque el endpoint MCP no tiene autenticación. La contraseña de MySQL y la clave de Gemini se leen del archivo `.env`. Para un despliegue real faltaría autenticar con OAuth2, usar TLS y un usuario de MySQL sin privilegios de administrador.

Paso 9. Pruebas

- **Lógica de negocio:** totales, descuento de stock, reversión completa, 12 compras simultáneas sobre 5 unidades, validaciones, propiedad del pedido y cancelación.
- **Protocolo:** un cliente MCP real registra un pedido por Streamable HTTP.
- **Chat:** memoria aislada por sesión, reinicio de la conversación, mensajes vacíos y fallos del modelo, con un modelo simulado.

Paso 10. Ejecución y conversación de prueba

Primero se inicia el servidor MCP y después el chatbot, porque este descubre las herramientas al arrancar:

```
cd supermercado\mcp-server
.\mvnw.cmd spring-boot:run
# en otra terminal
cd supermercado\chatbot
.\mvnw.cmd spring-boot:run
```

En `http://localhost:8081` se siguió esta conversación:

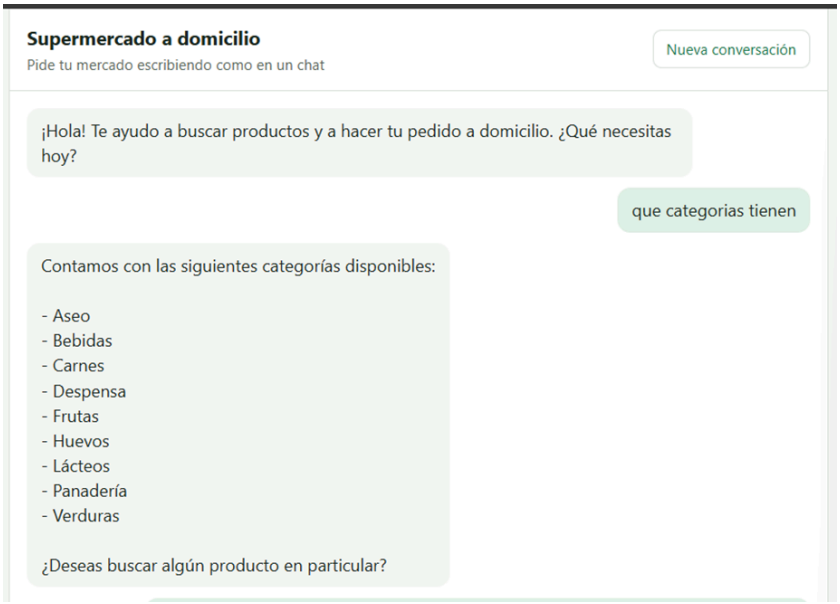
1. "¿Qué categorías tienen?": el chatbot invocó `listarCategorias` y mostró las 9 categorías (Figura 13).
2. "Necesito arroz y 2 litros de leche": invocó `buscarProductos` y mostró cada producto con su precio.
3. Se dieron la cantidad, el nombre, el teléfono y la dirección: el chatbot mostró un resumen con un total de $18.800 y pidió confirmación, sin registrar todavía el pedido.
4. "Sí, confirmo": invocó `crearPedido` e informó el número de pedido (1) y el total.

El resultado se verificó en MySQL:

```
SELECT id, cliente_nombre, estado, total FROM supermercado_mcp.pedido;
SELECT nombre, stock FROM supermercado_mcp.producto WHERE nombre IN ('Arroz blanco',
'Leche entera');
```

En MySQL quedó el pedido 1 en estado `RECIBIDO` con un total de 18.800, y el stock se descontó: Arroz blanco quedó en 118 unidades y Leche entera en 148.

En una prueba posterior, el chatbot respondió "No pude responder en este momento. Revisa que el servidor MCP y la clave de Gemini estén configurados e inténtalo de nuevo". Es el manejo de errores previsto: si falla la llamada a Gemini o al servidor MCP, el chatbot muestra un mensaje en lugar de romper la conversación. La causa más probable es una saturación temporal del modelo, como el error 503 "high demand" observado durante el laboratorio; si se repite, el modelo se puede cambiar con `GEMINI_MODEL` en `.env`.



[Figura 13. Chatbot mostrando las categorías]

8. Resultados de las pruebas y problemas encontrados

Todas las pruebas de la tabla de la guía se cumplieron. La única diferencia es que el Inspector muestra 9 herramientas en lugar de 7, por las dos agregadas en las actividades 10 y 11.

Prueba	Acción	Resultado esperado	Resultado obtenido	Evidencia
Conectividad	Iniciar Spring Boot	Servidor activo en 8088	Servidor activo en 8088, sin errores	Figura 1
Negociación	Enviar `initialize`	Respuesta MCP válida	Respuesta con capacidades y `tarea-mcp-server` 1.0.0	Figura 2
Descubrimiento	Listar tools en Inspector	7 herramientas visibles	9 herramientas visibles	Figuras 3 y 4
Lectura	`listarPendientes`	Solo tareas no completadas	Tareas 1 y 2	Figura 5
Creación	`crearTarea`	Nueva fila en MySQL	Tarea 4 creada	Figura 6
Actualización	`marcarCompletada`	`completado = true`	Tarea 4 con `completado = 1`	Figura 7

Prueba	Acción	Resultado esperado	Resultado obtenido	Evidencia
Claude Desktop	Pedir tareas pendientes	Respuesta basada en datos reales	Lista correcta y tarea 5 creada	Figuras 8 y 9
Google ADK	Consultar tareas desde el agente	El agente invoca herramientas MCP	Invocó `listarPendientes` y `crearTarea` (tarea 7)	Figuras 10 a 12
Ejercicio propuesto	Pedido completo por el chatbot	Pedido registrado y stock descontado	Pedido 1 registrado tras confirmación y stock descontado	Figura 13

Problemas encontrados y soluciones

Problema	Causa	Solución
Maven no compilaba el proyecto	`JAVA_HOME` apuntaba al JDK 17 y el proyecto exige Java 21	Se cambió `JAVA_HOME` al JDK 21 con `setx`
`clean install` del agente fallaba con "Unable to find main class"	El POM de la guía no indica una clase principal	Se agregó `AdkWebServer` como `mainClass` en `spring-boot-maven-plugin`
La guía menciona los puertos 8088 y 8888	Inconsistencia de la guía	Se usó 8088, el puerto del proyecto base y de la mayoría de los pasos
Los nombres de las herramientas del proyecto base no coincidían con la guía	El proyecto base usaba `listTareas`, `markComplete`, etc.	Se renombraron para coincidir con las pruebas de la sección 8.3
El agente ADK no respondía ni invocaba herramientas (error 500)	Google ya no ofrece `gemini-2.5-flash` a cuentas nuevas (404) y `gemini-3.8-flash` estaba saturado (503 "high demand")	Se cambió a `gemini-3.5-flash` y el modelo quedó configurable con `GEMINI_MODEL` en `.env`

El caso del agente ADK muestra la importancia de leer los registros: la interfaz web solo mostraba que no había respuesta, y la causa real (el modelo retirado y luego saturado) solo apareció en el log del servidor.

9. Conclusiones

- **MCP desacopla las herramientas de los clientes.** El mismo servidor de tareas sirvió sin cambios a Claude Desktop y al agente ADK, con modelos de proveedores distintos. Ante peticiones equivalentes, ambos eligieron las mismas herramientas. La integración se escribe una sola vez y la reutiliza cualquier cliente compatible.
- **La descripción de una herramienta es parte de su interfaz.** El modelo no ve el código, solo el nombre, la descripción y el esquema. Una descripción que dice qué devuelve y cuándo usarla, como la de `contarTareas`, hace que el modelo elija la operación correcta.
- **Las reglas de negocio deben vivir en el servidor.** La validación de fechas, el cálculo de precios y totales y el descuento atómico de stock no dependen de que el modelo se comporte bien. El modelo solo propone argumentos; el servidor decide si son válidos.
- **Las acciones con impacto requieren confirmación.** El chatbot del supermercado muestra un resumen y espera la confirmación del cliente antes de invocar `crearPedido`, y el agente confirma el identificador antes de eliminar.
- **Exponer herramientas por HTTP sin autenticación es un riesgo real.** Cualquier equipo que alcance el puerto puede invocarlas sin un modelo de lenguaje. Restringir la escucha a `127.0.0.1` es la primera defensa; un despliegue real necesita OAuth2, TLS y mínimo privilegio.
- **Los secretos se gestionan fuera del código.** El archivo `.env`, ignorado por Git, junto con la plantilla `.env.example`, permite que otra persona clone el repositorio y lo configure sin que las credenciales se publiquen.
- **Las dependencias externas cambian y los registros son la herramienta de diagnóstico.** El modelo que indicaba la guía ya no estaba disponible para cuentas nuevas. El error solo era visible en el log del servidor, y dejar el modelo configurable evitó tener que cambiar el código cuando un modelo se retira o se satura.

10. Bibliografía

Botia, D. J. L. (2026). *Construcción e integración de un MCP Server: Spring Boot + Spring AI + MySQL + Claude Desktop + Google ADK* [Guía de laboratorio]. Departamento de Ingeniería de Sistemas, Universidad de Antioquia.

mesasg. (2026). *MPC-Server* [Repositorio de código]. GitHub. https://github.com/mesasg/MPC-Server

URLs

Recurso	Enlace
Repositorio	https://github.com/mesasg/MPC-Server
Proyecto base del servidor MCP	https://github.com/diegobotia/labmcp
Proyecto base del agente ADK	https://github.com/diegobotia/agentemcp